LIGHTNING TALK

When Your Search Outgrows Your Platform

Real infrastructure patterns from a production system serving 700+ users.

Rebecca Burch

Software Engineer II, Texas A&M Transportation Institute

Elastic Austin User Group · April 16, 2026

LinkedIn

Rebecca Burch

Software Engineer II, Texas A&M Transportation Institute

DAY JOB

Backend infrastructure, data pipelines, and the search system this talk is about

BACKGROUND

Biomedical sciences (neuroscience) turned software engineer

BUILDING

Cascade, an agentic AI runtime with memory architecture. Wandr ATX, 3,200 users at SXSW.

Austin, TX

700+ people, one search box

WordPress

Backend platform with native search

700+

Employees in an external API

1 box

Find anyone by name, org, or specialty

Small agency, no budget for Elastic. WordPress has search built in. How hard could it be?

It started so simply

simple-search-shortcode.php — live API call, no index, no ranking.

```
// edge cases: two-letter last names
$short_names = [
    'md', 'le', 'yu', 'wu', 'ho', 'oh', 'an',
    'la', 'do', 'so', 'um', 'ng', 'ko', ...
];





// from simple-search-shortcode.php
```

A hardcoded list of edge cases is the platform telling you it can't.

Custom indexing layers

When the platform doesn't index what you need, you build a shadow index.

```
register_post_type(
  'staff_directory',
  [ 'public' => false, ... ]
);

// This CPT exists solely for
// Relevanssi search indexing.
// The actual UI displays data
// directly from API/transients.
```

WHAT THIS REALLY IS

Every employee gets a shadow document inside WordPress, with one job: be searchable. The UI never reads it.

ELASTICSEARCH PRIMITIVE

Native document index. Mapping templates. No platform contortion required.

Incremental sync pipelines

Reindexing 700 people every run is too slow. So you write change detection.

```
function content_hash(
  $name, $org, $title,
  $specialties, $photo, $resume
) {
  return md5(serialize([
    $name, $org, $title, ...
  ]));
}



// generate_employee_content_hash() + offset in wp_options
```

WHAT THIS REALLY IS

Hash compare, skip if unchanged, resume from offset. You've written change detection inside your application layer.

ELASTICSEARCH PRIMITIVE

_seq_no, partial updates, and the bulk API. Already there, already correct.

Hand-tuning relevance

Results aren't good enough. So you reach into the scoring loop.

```
// boost_specialty_matches()
$match->weight =
  $match->weight * 200;

// then for team pages...
$match->weight =
  $match->weight * 250;




// magic multipliers, no principled basis
```

WHAT THIS REALLY IS

Why 200? Why 250? Trial and error. You've reinvented field boosting, one PHP filter at a time.

ELASTICSEARCH PRIMITIVE

BM25 with named field boosts. function_score, dis_max, explain API. Knobs that mean something.

Reconciliation logic

What happens when an employee leaves? You write a set-difference loop.

```
$processed_pids = [];
foreach ($api_employees as $e) {
  $processed_pids[] = $e->pid;
  upsert_post($e);
}

// draft anything not seen this run
draft_missing($processed_pids);




// from the WP-CLI index-staff command
```

WHAT THIS REALLY IS

Track every PID seen, then diff against published posts, then draft the orphans. Manual reconciliation.

ELASTICSEARCH PRIMITIVE

delete_by_query, or an alias swap to a freshly built index. Two lines, atomic.

Rate-limiting your own pipeline

The API can't handle 700 rapid calls. So you sleep.

```
usleep(100000);
// 100ms throttle to avoid
// API rate limits

// appears in 3 places:
//   cron indexer
//   WP-CLI staff indexer
//   picture sync
```

WHAT THIS REALLY IS

You're managing concurrency by sleeping. In three different files. That's a queue, written badly.

ELASTICSEARCH PRIMITIVE

Bulk ingest with backpressure, async pipelines, and ingest nodes built for this exact problem.

THE REALIZATION

I wasn't patching search.
I was building a search engine.

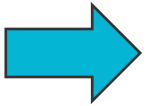
And I kept building it. Elastic wasn't in the budget. The patches were.

What Elastic would have given me

WHAT WE BUILT

WordPress + Relevanssi

- staff_directory shadow CPT
- MD5 hash + offset cron
- Magic weight multipliers
- PID reconciliation loop
- usleep() in three files



WHAT ELASTIC SHIPS

Out of the box

- Native index, real mappings
- Bulk + update-by-query
- BM25 + named field boosts
- Alias swaps for atomic cutover
- Backpressure via bulk ingest

We didn't migrate. The cost wasn't justified for our size. But knowing the patterns made me a better engineer.

WHAT IT COST US

5 custom subsystems to maintain

0 frameworks for measuring relevance

1 engineer who really understood the search stack

700 employees depending on it

It worked. It was expensive in the ways that don't show up on a budget line.

Three signs you're rebuilding a search engine

01

You're writing your own indexer

Shadow CPTs, flatteners, hand-rolled mapping. If your application code is making things searchable, you're maintaining a search engine.

02

Your sync is held together by cron and sleep

Polling, hashing, throttling, draft-the-orphans. That's a queue and a pipeline. You're running both, without the tooling.

03

You can't explain why a result ranked first

Magic multipliers (* 200, * 250) mean no one can debug relevance. Search becomes the thing the team is afraid to touch.

Q & A

Thanks, Austin.

Ask me about: search architecture, what these patterns cost us, or how I'd make the case for Elastic if I were starting over.

Rebecca Burch

linkedin.com/in/rebecca-burch

Austin, TX · Backend Infra & Search Architecture

LinkedIn

Wandr ATX

Cascade